

columns compose an one-hot vector indicating the corresponding polyomino, and the last n columns are marked with 0 or 1 depending on whether the corresponding unit is filled by that polyomino. Then finding a set of polyomino arrangements in a rectangle equals to find a set of rows containing exactly one 1 in each column. Another adaptable problem is the boolean satisfiability problem (commonly known as SAT), for which efficient solvers exist [ES03, GN07]. A tiling problem can be converted to SAT by introducing a boolean variable for each possible arrangement of each piece, and then adding clauses comprising of those boolean variables that ensure at least one arrangement of each piece is achieved, while avoiding conflicts between arrangements of one piece or two different pieces.

Given the size of a rectangle and polyominoes to be fit, multiple corresponding solutions are generated by applying those algorithms. Then in the question generation stage, we randomly mask several polyomino pieces in the rectangle, and generate a question answer(QA) pair for each masked polyomino. Finally the rectangle and each polyomino piece are rendered with emoji squares.

$$\begin{array}{c}
 P1 \\
 \vdots \\
 Pk
 \end{array}
 \begin{array}{c}
 C_1 \dots C_k \quad C_{k+1} \dots \dots \dots C_{n+k} \\
 \left[\begin{array}{cc|cc}
 1 \dots \dots 0 & 1 & 0 \dots 0 & 1 \\
 1 \dots \dots 0 & 0 & 1 \dots 1 & 0 \\
 \vdots & & & \vdots \\
 0 \dots \dots 1 & 1 & 1 \dots 0 & 0 \\
 0 \dots \dots 1 & 0 & 0 \dots 1 & 1
 \end{array} \right]
 \end{array} \quad (9)$$

A.3 Visual Data Rendering

After gathering the textual dataset of 2D square grid, we generate the corresponding visual dataset by drawing text onto an image. Specifically we adopt color emojis for a fair comparison as they're more visual friendly to a multimodal model.

A.4 Dataset Details

Data distribution among various difficulty levels for visual navigation tasks and visual tiling tasks are provided in Table 4 and 5. It provides flexible difficulty control across different tasks. For visual tiling task, the difficulty is controlled by the number of masked polyomino pieces. As the number increases, the more spatial arrangements LLMs need to consider. Regarding the visual navigation task, as illustrated in figure 2, we use the number of roads k to control difficulty, which is corresponding to the size of the map.

Task	K Step						Total
	2	3	4	5	6	7	
Route Planning	8	16	32	64	128	248	496
Next Step Prediction	8	32	96	256	640	1488	2520

Table 4: Data distribution of visual navigation dataset with the total navigating step of k indicating difficulty level. The reason why the number of generated map is slightly lower than 2^{k+1} for $k > 5$ is explained in Appendix A.1.

	Mask count		Total
	2	3	
Configuration	248	124	376
QA Instance	489	307	796

Table 5: Details of visual tiling dataset. Some QA instances are discarded when multiple solutions exist and all answers are correct.

B Examples

For visual navigation and visual tiling tasks, the structured input template is comprised of task instruction, input parameters and prompt of specific setting.

B.1 Visual Tasks

Task instructions and responses of each visual task under setting GPT-4 VoT are provided as following:

- Route Planning Task instruction in Figure 6, response in Figure 12 .

Visualize the state after each reasoning step

First, let's look at the target rectangle and identify the empty squares (□) where we need to fit the polyominoes:



We have two polyominoes to fit into this rectangle: Tetromino I (■) and Tetromino T (■). The Tetromino I can only fit in the bottom row since it's the only place with four consecutive empty squares, either horizontally or vertically. Let's place the Tetromino I (■) in the bottom row:



.....
We can see that Variation 2 of Tetromino T (■) fits perfectly into the remaining empty squares, as it matches the T shape formed by the empty squares. Let's place Variation 2 of Tetromino T (■) into the target rectangle:



The correct variation of Tetromino T to fit into the empty squares is Variation 2.

Answer: A. 2

Let's think step by step.

First, let's look at the empty spaces in the target rectangle. We have a 4x1 space at the bottom and three single squares above it. The Tetromino I (■) can only fit in the 4x1 space at the bottom, so we'll use Variation 2 of the Tetromino I to fill that space.

Now, let's consider the Tetromino T (■) variations. We have three single empty squares that are not in a straight line, so Variation 2 of the Tetromino T, which is a straight line with a protrusion, will not fit. However, Variation 3 of the Tetromino T has a configuration that can fit into the three remaining empty squares with one square protruding to the side, which matches the layout of the empty squares.

Therefore, the correct variation of Tetromino T to fit into the remaining empty squares is:

B. 3

The final configuration would look like this:



(a) Common behaviors of VoT and CoT prompting in visual tiling task, with the overall track rate of 87.1% and 57.4% respectively.

Visualize the state after each reasoning step

To navigate from 🏠 to 🏢, we need to find the path that avoids obstacles (🚧) and follows the roads (□). Here's the step-by-step navigation:

1. Move down from 🏠 to the end of the continuous road.



After the move:



2. Move left from the current position to the office 🏢.



After the move:



The 🏠 has now reached the 🏢, completing the navigation task.

Visualize the state after each reasoning step

To navigate from 🏠 to 🏢 on the provided map, follow these instructions:

1. Move down to the end of the continuous road.

2. Move right to the end of the continuous road.

3. Move up to reach the destination 🏢.

Here's the visualization after each step:

Starting point:



After Step 1 (Move down):



After Step 2 (Move right):



After Step 3 (Move up):



(b) Route planning task is sensitive to prompts. After deleting the word "reasoning" from the VoT prompt, final answer and state is derived without conditioning on state visualization in many cases. The left example provides a correct solution with imperfect intermediate visualization, while the right example provides a wrong solution.

Figure 8: Example cases of visual state tracking behaviors in different tasks. We provide side-by-side comparisons to show how visual state tracking is influenced by prompt, where bold texts on the top are prompts to compare. Case on the left exhibits visual state tracking in an interleaved manner, where next state is conditioned on visualization of previous states. Case on the right disentangles verbal reasoning and state visualization, which leads to a drop in tracking rate and task performance.